

'Data Structures'

"organising the data"

s.t it is easier for the task at hand.

+ algorithms which makes the task
"efficient"

Time Complexity -
↳ Number of execution steps.
Space Complexity.

```
for (int i=0; i < 3, ++i) {  
    printf("%d", &i);  
}
```

i=0 ; 0
i=1 ; 1
i=2 ; 2
i=3

"Big-Oh Notation."

↳ worst case running time.

n = input

O()

3

$\frac{n^2 + n + 1}{2}$
↳ $O(n^2)$

```
for (int i=0; i < n; ++i)  
{  
    printf("%d\n", &i);  
}
```

$n+1$

$O(n+1)$: $O(n)$ → $O(n)$

```

for (int i=0; i<n; ++i) → i=0; i:=2
{
    for (int j=0; j<m; ++j)
    {
        printf("%d-%d\n", &i, &j);
    }
}

```

n times

$$\underbrace{m + m + m + \dots + m}_{n \text{ times}} = mn = O(mn)$$

```

for (int i=0; i<m; ++i) → i=0; i:=2
{
    for (int j=0; j<i; ++j)
    {
        printf("%d-%d\n", &i, &j);
    }
}

```

$$0 + 1 + 2 + \dots + m-1$$

$$\frac{(m-1) \times m}{2} = \frac{m^2 - m}{2} = \frac{m^2}{2} - \frac{m}{2} = O(m^2)$$

```

for (int i=1; i<n; i*=2) i=1; i:=2
{
    printf printf("%d\n", &i); → 2...
}

```

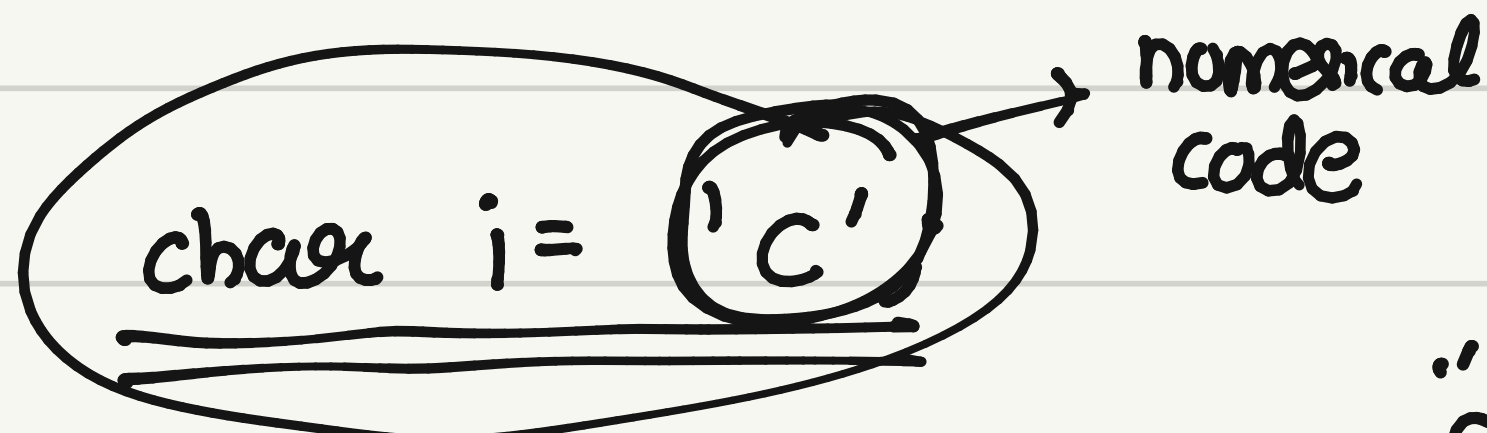
$\underline{x} = \log_2(n)$

$$\begin{aligned}
 &1 + 1 + \dots \\
 &\underline{2^x} \leq n \quad \downarrow \quad \underline{2^x = n} \quad \boxed{n = 2^x} \\
 &\log_2(2^x) = \log_2(n) \\
 &\underline{x = \log_2(n)}
 \end{aligned}$$

Big-Theta $\Theta(n)$ $\rightarrow$
 Big-Omega $\Omega(n)$

Pointers

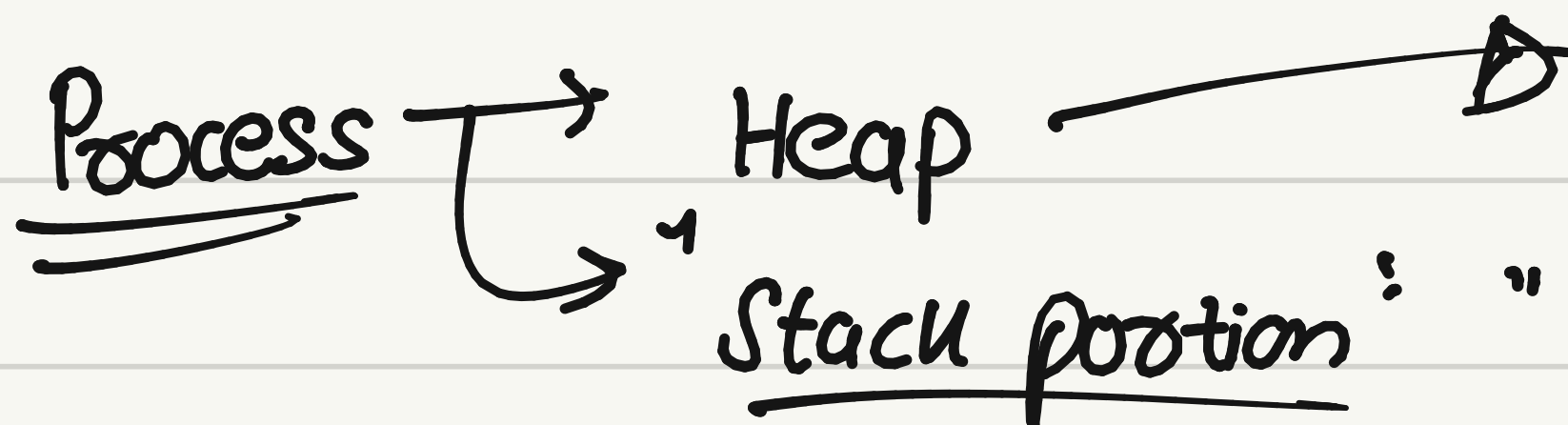
"variable"



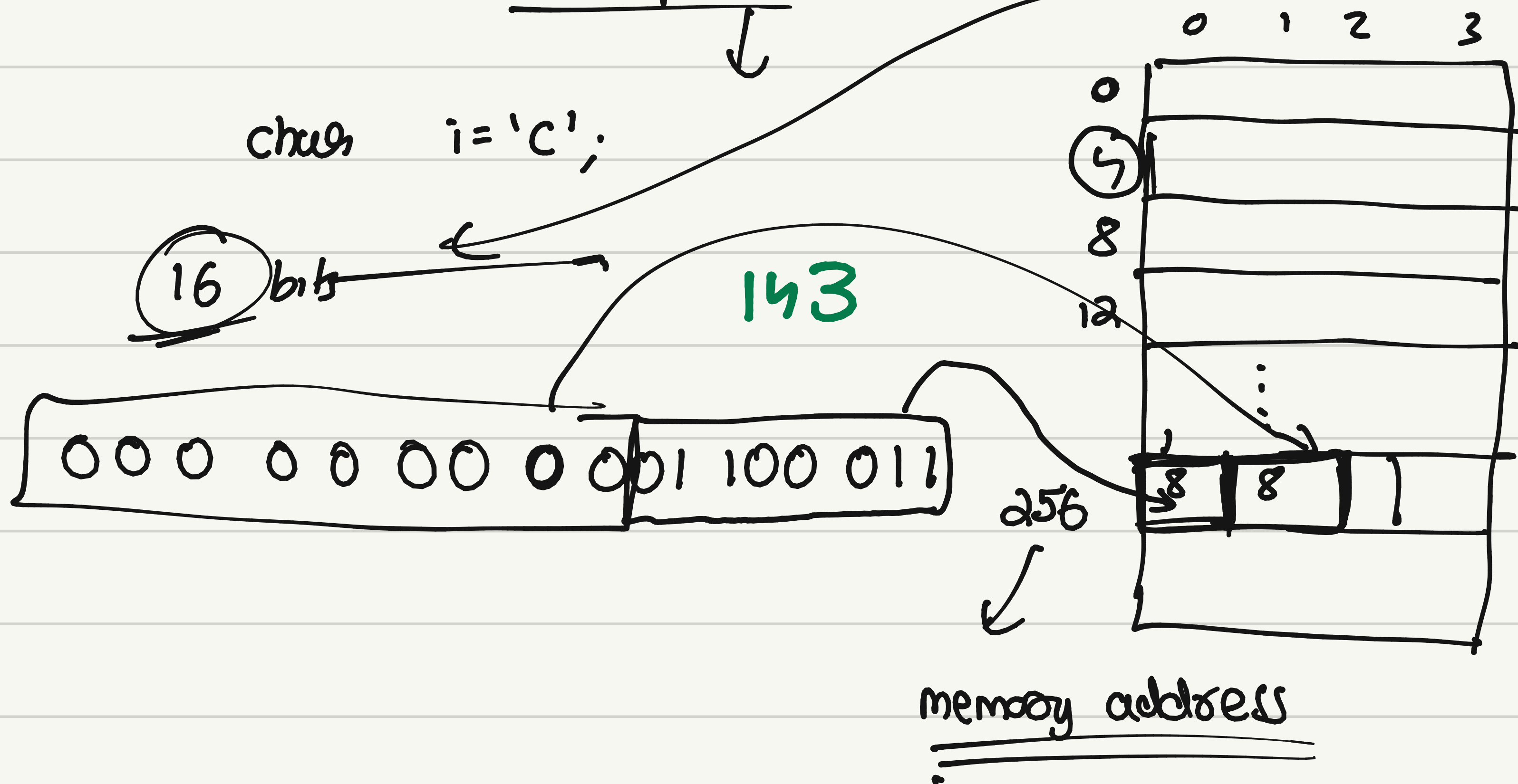
$\leftarrow$ int
 char $\rightarrow$ "c"
 float; $\rightarrow$ '2';

ASCII

"Global variable"



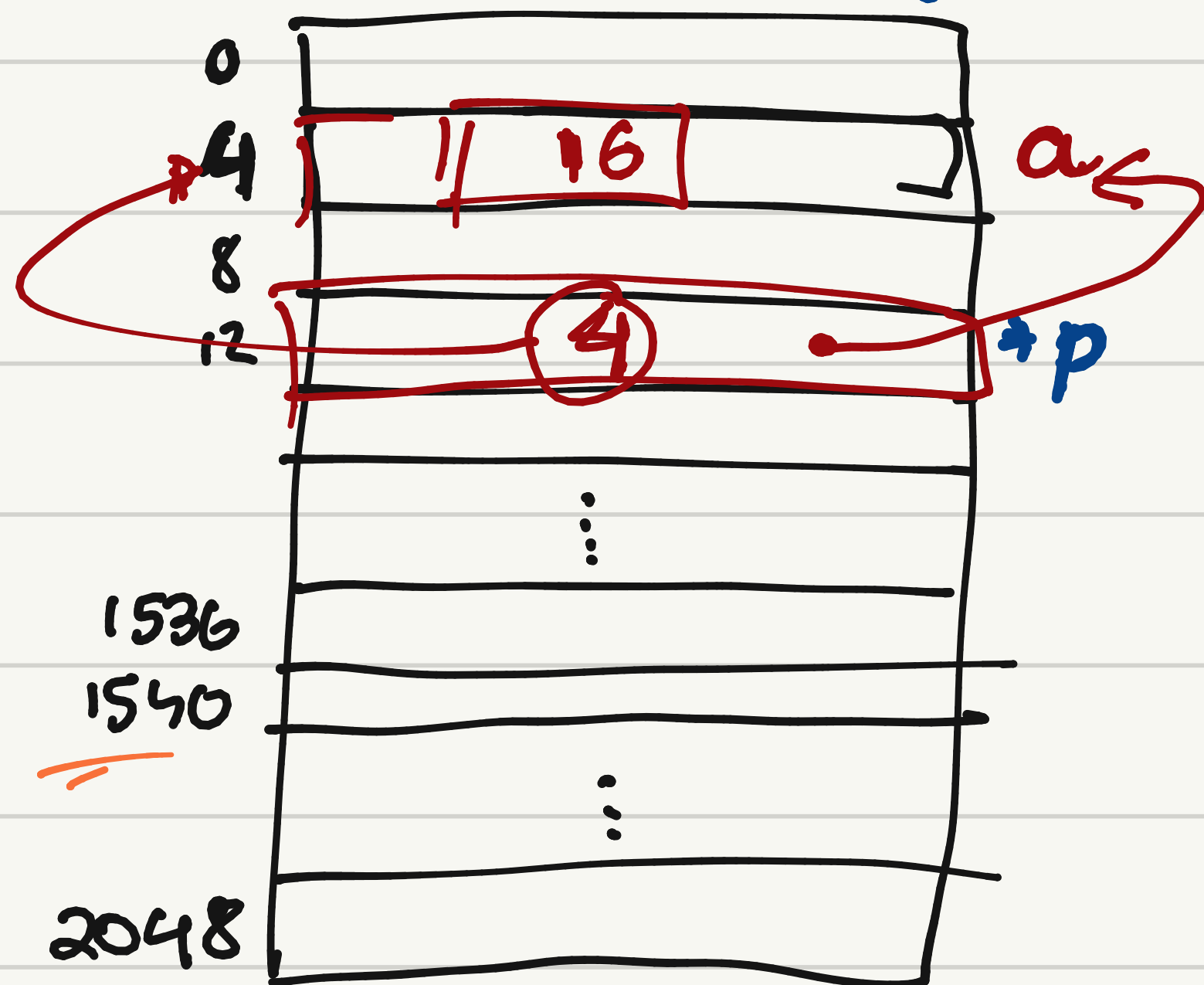
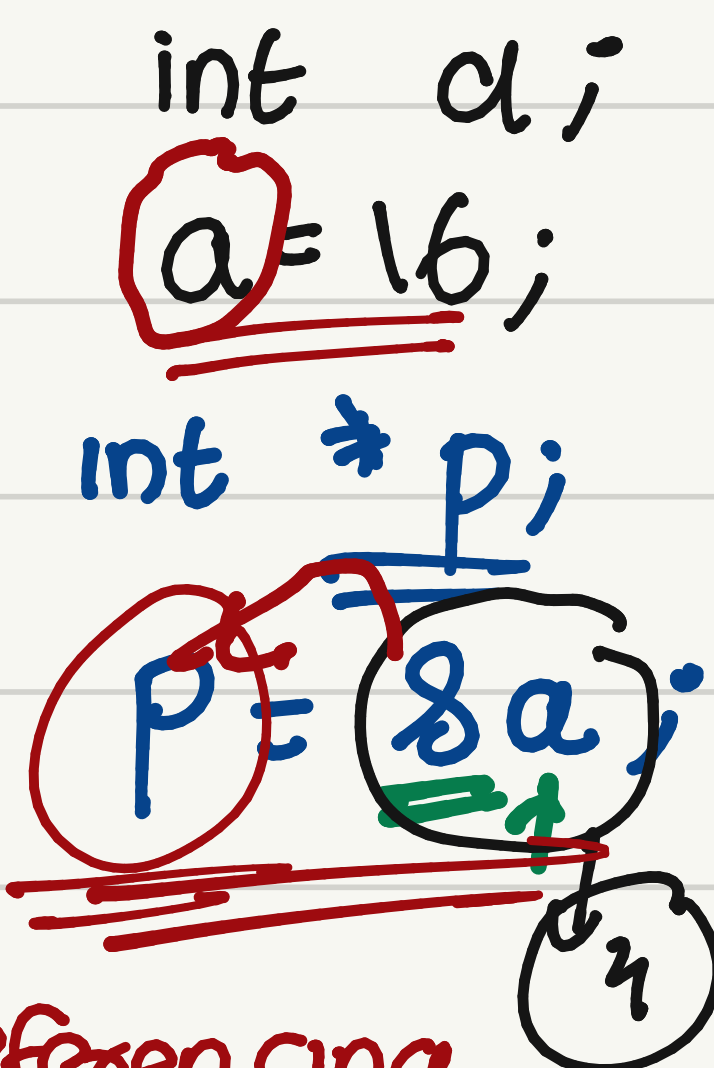
char - "2 bytes"



Pointer

"variable" $\rightarrow$ store memory address;

"int" $\rightarrow$ 4 bytes
 "address" $\rightarrow$ 4 bytes.



$*$ $\rightarrow$ Dereferencing operator
 $*p$
 $*(4)$
 $*p = 16$
 (p)

$\text{int } a;$
 $a = 343;$
 $\text{int } *p = \&a;$

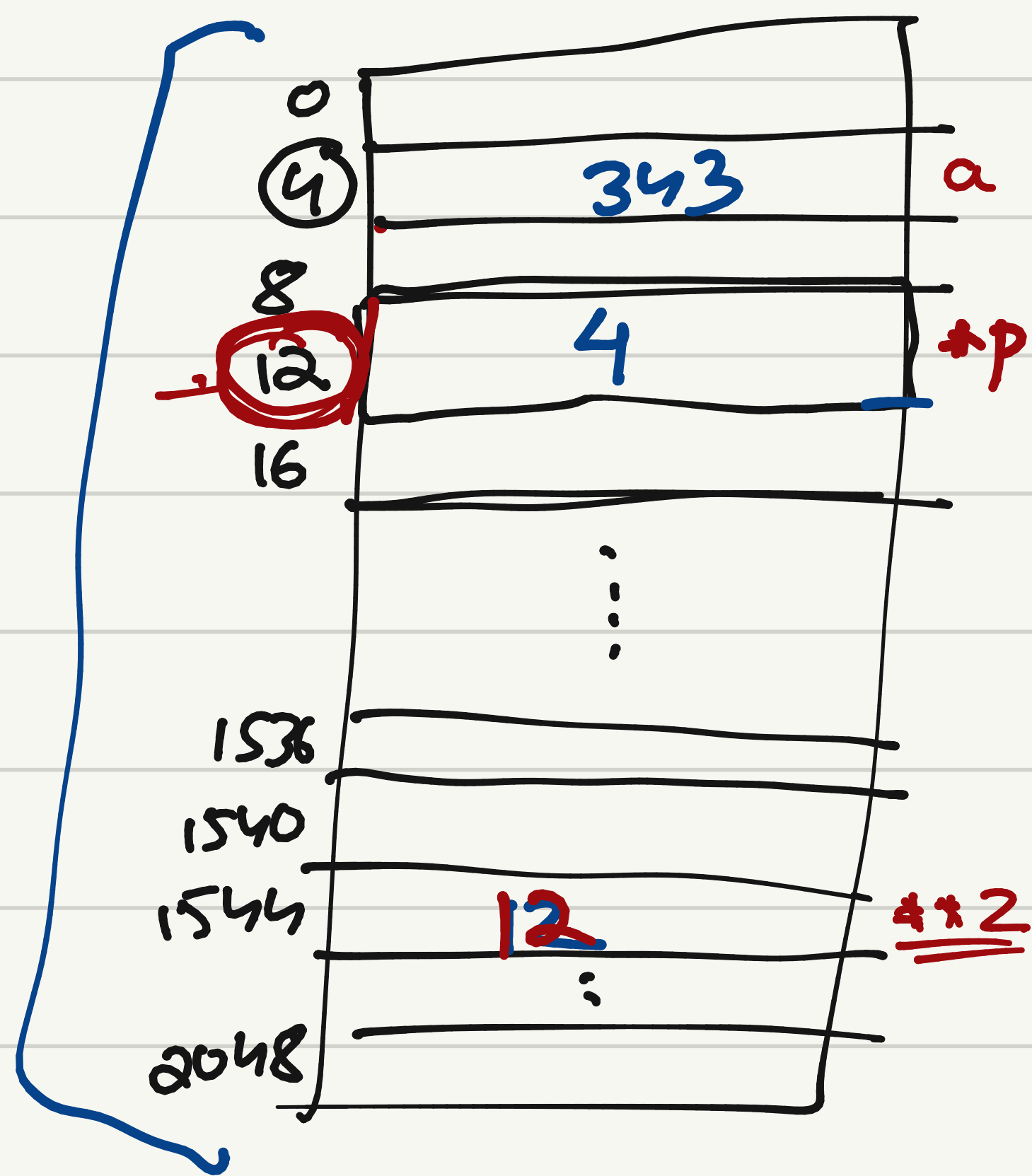
$\text{int } **z;$
 $z = \&p;$

$*z$

$+ (12) \rightarrow$

4

$$**z = *(*z) = *(4) = 343$$



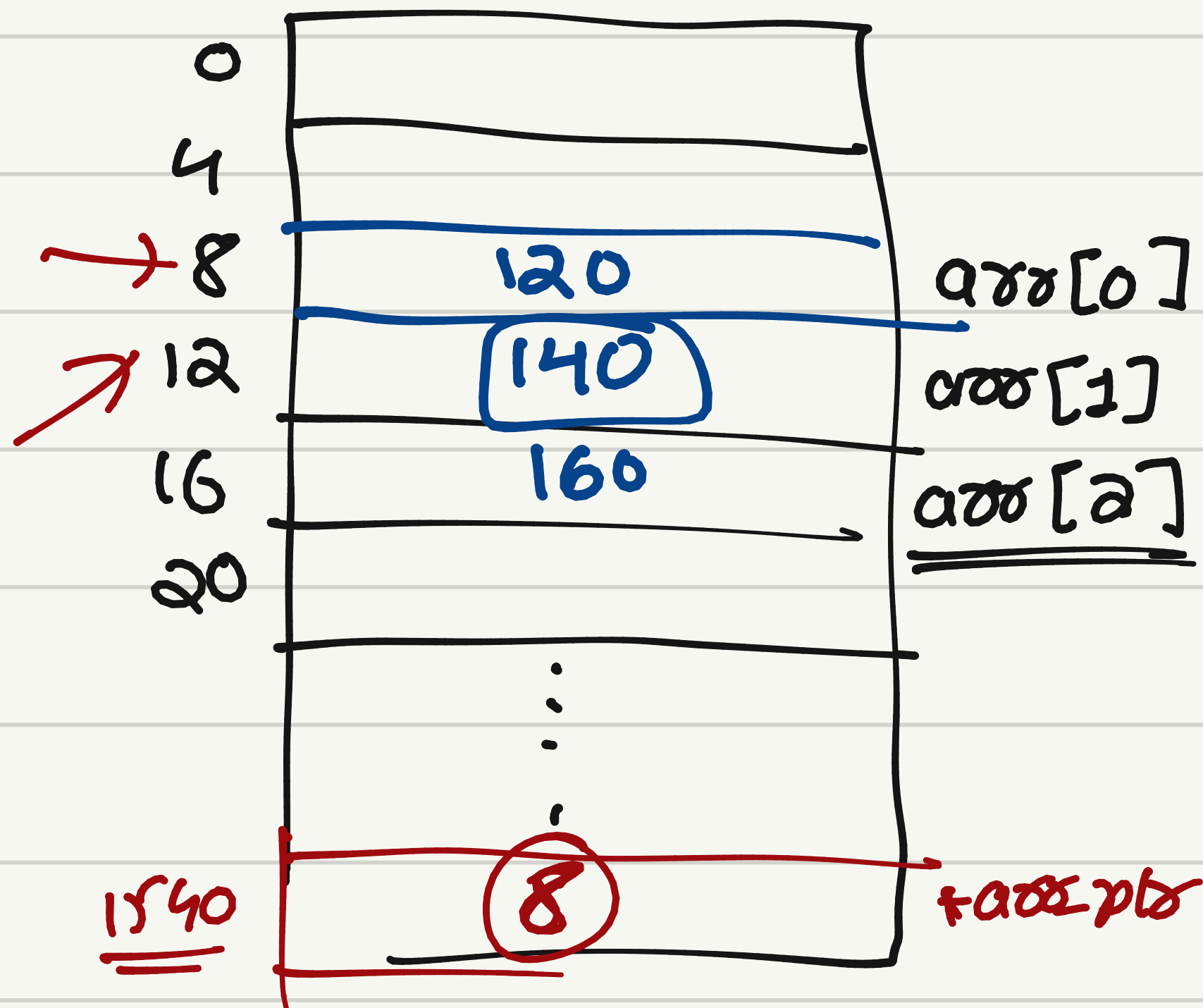
Array

Collection of elements of same type

↳ elements are stored in contiguous memory locations

$\text{int } arr = \{120, 140, 160\};$

arr → address
 $arr[0]$ → value
storing element



arr_ptr

4

address occupies 4 bytes

$+1$

$8 + 1 = 9$ (crossed out)

$8 + 4 = 12$

Adding the number of bytes taken to store an address

$8 + 4 = 12$

$*(12) =$

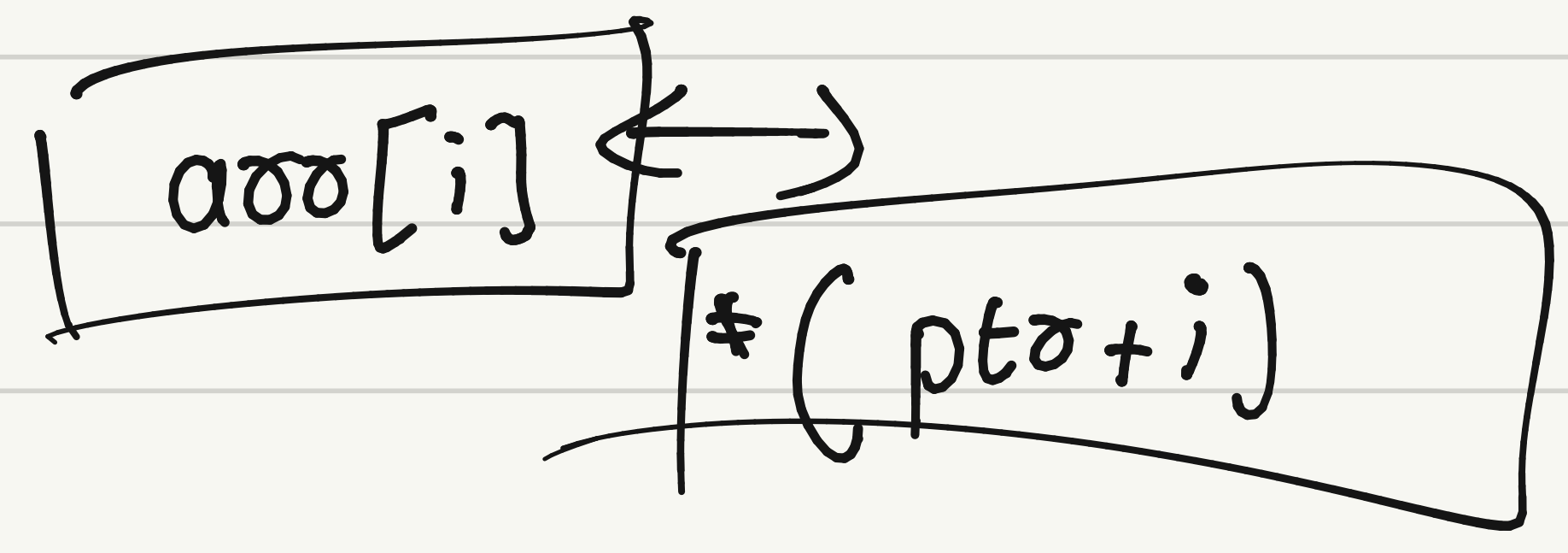
$*(arr_ptr + 1) = 12$

4 bytes

$arr_ptr + 2$

$arr[1]$

arr[] = { };
 int * ptr = arr;



n-dimensional arrays $\Rightarrow$ Tensor

Matrix = 2D array

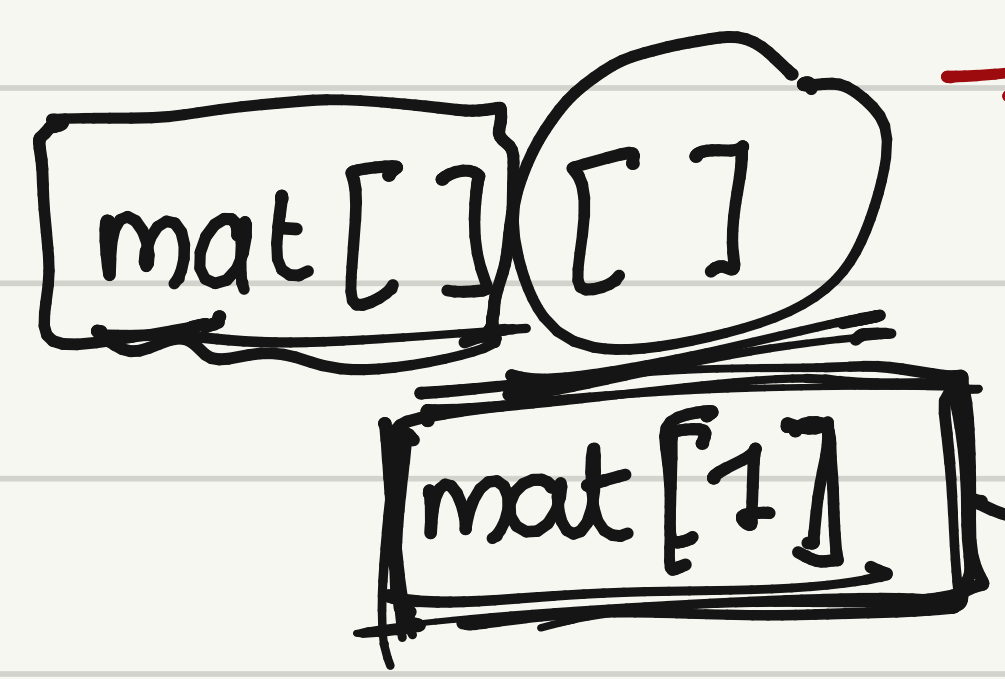
int mat[3][3] = {
 $\rightarrow$ {40, 50, 60},
 $\rightarrow$ {400, 450, 480},
 {520, 560, 580}
 };



arr ⑧
arr[]

mat[0]	{	0	40
		4	50
		8	60
mat[1]	{	12	400
		16	450
		20	480
mat[2]	{	24	520
		28	560
		32	580

mat[1]
* (8mat[1])

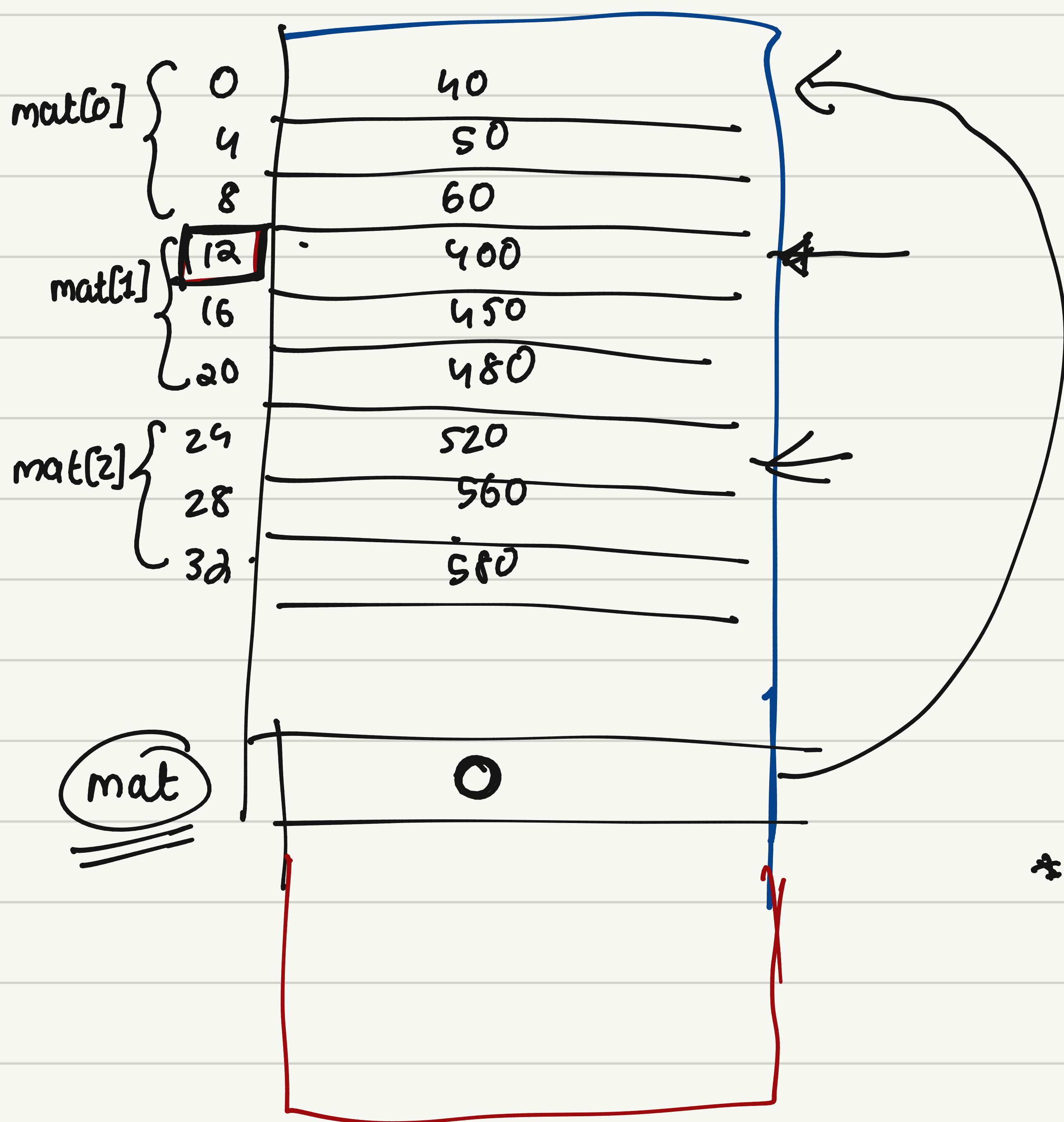


Address

* (mat[1]) = value at starting of second row

arr[i]

$$\text{int } *a = \text{arr};$$
$$\text{arr}[i] = \underline{\underline{* (a+i)}}$$



{ {40, 50, 60},
{400, 450, 480},
{520, 560, 580}}
}

mat[1][1]

* (mat+1)

* (* (mat+1) + 1)

mat[1] / mat[2]

* (mat+1)

mat → row

3

3x4 = 12

* (mat+1)

mat[i][j]

* (* (mat+i) + j)

Take the number of elements in row 0,
incrementing the address by
no of elements * address size of
1 element

0 + 3x4 =

* (* (mat+i))

mat[1][0]

* (* (mat+2))

mat[2][0]

* (* (mat+2) + 0)

mat[2][0]

Structures in C

group of related data;

class ; Java, C++;

struct Student {

int rollno;

char name[100];

float height;

struct Student *next;

}

struct Student s1,

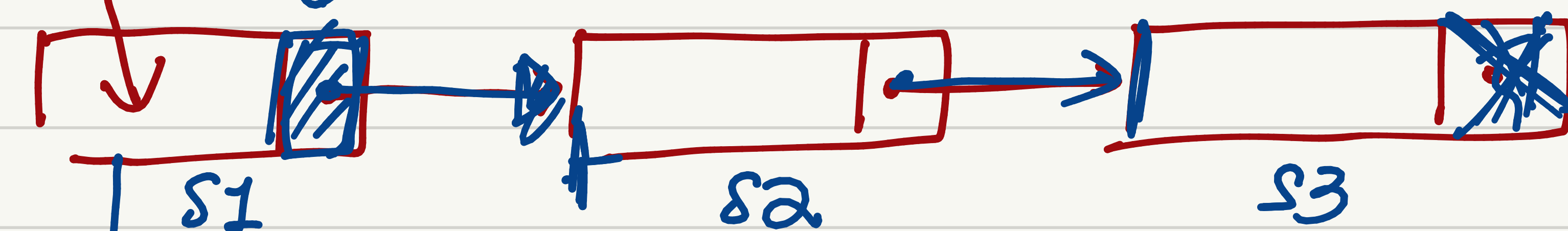
s1.rollno = 1;

s1.name = "Jacob";

s1.height = 170.0;

Linked List → "chain" of records

↳ Each record contains a pointer to another record of same type



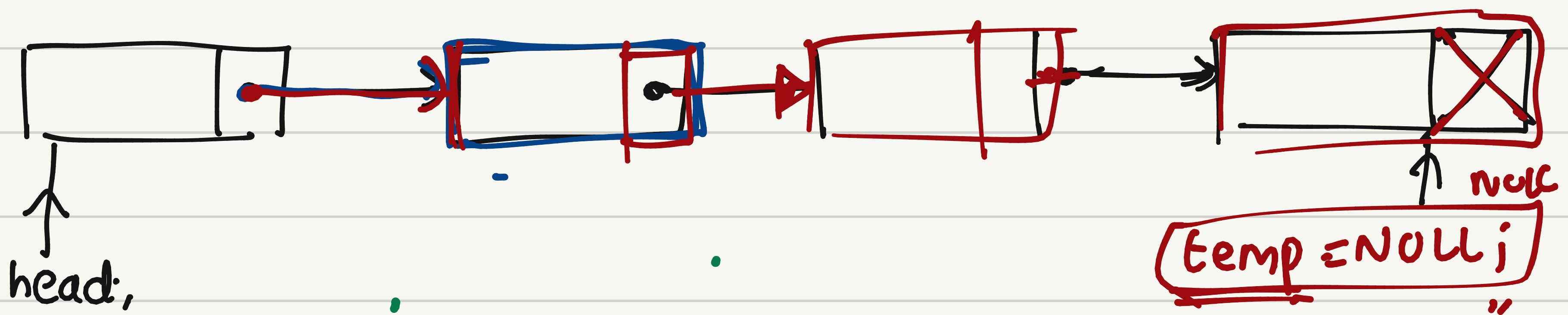
s1.next = s2;

s2.next = s3;

null pointer

"s1;"

s1;



temp = NULL;

while ("temp != null")
{

struct Student * head ;
struct Student * temp ;

✓ ✓ printf(" %s", temp->name);

temp = head;

temp = temp->next;

}

Inserting a new record to a linked list

struct Student S4;
S4.rollno = " ",

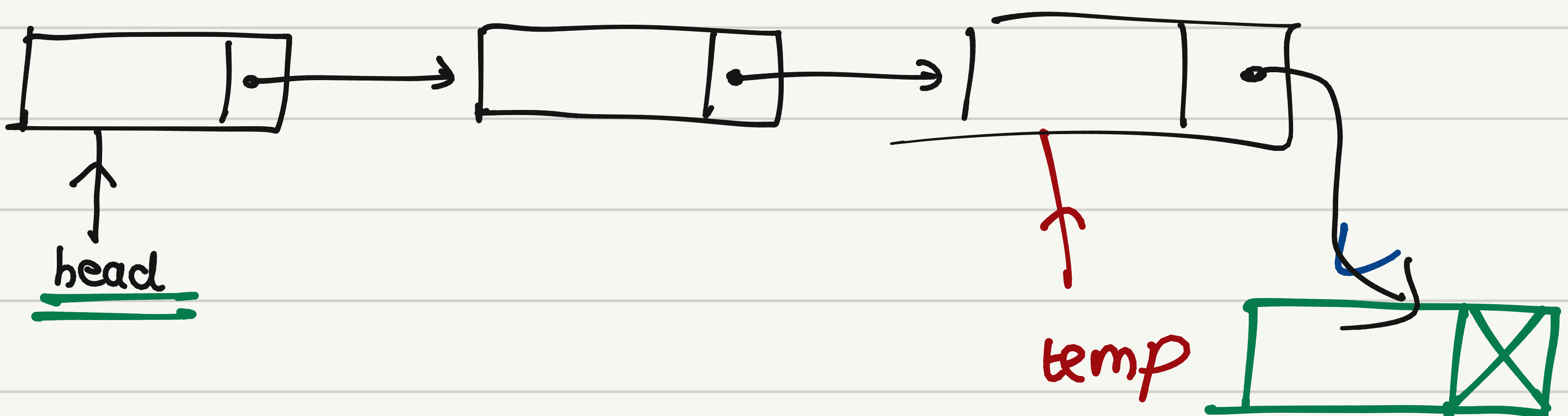
malloc

4
struct Student * S4 = (Student*) malloc (sizeof(Student));

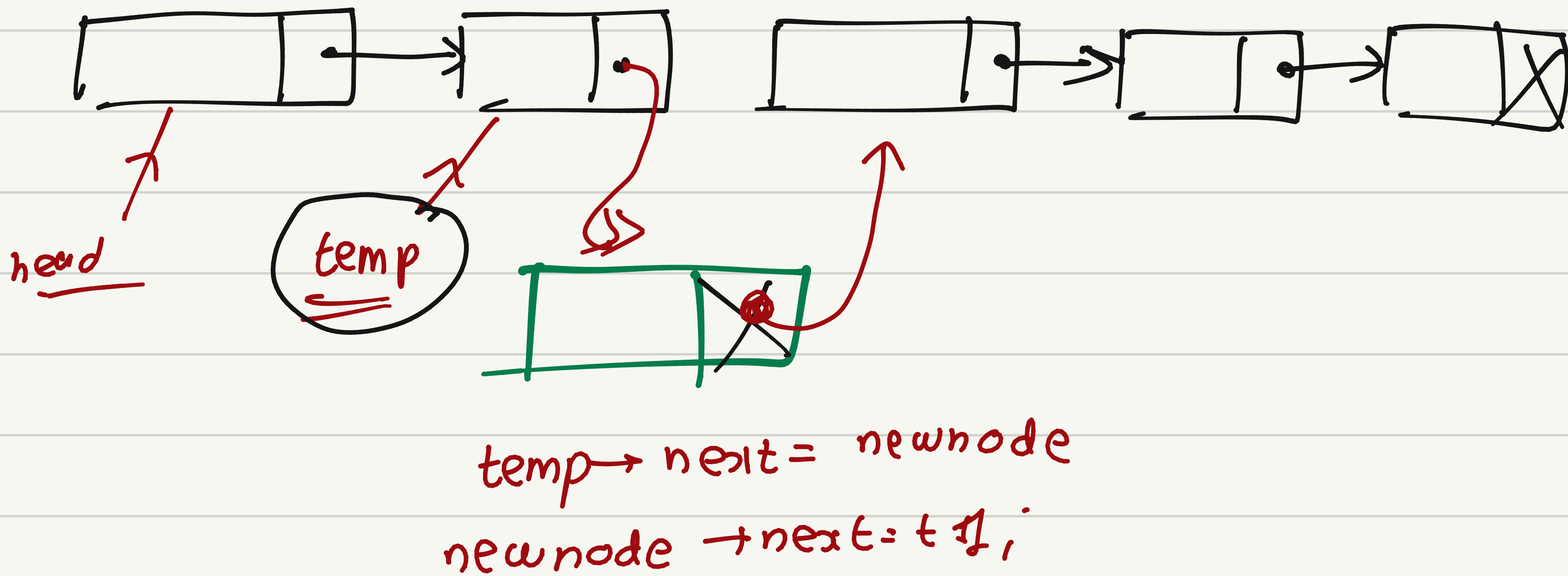
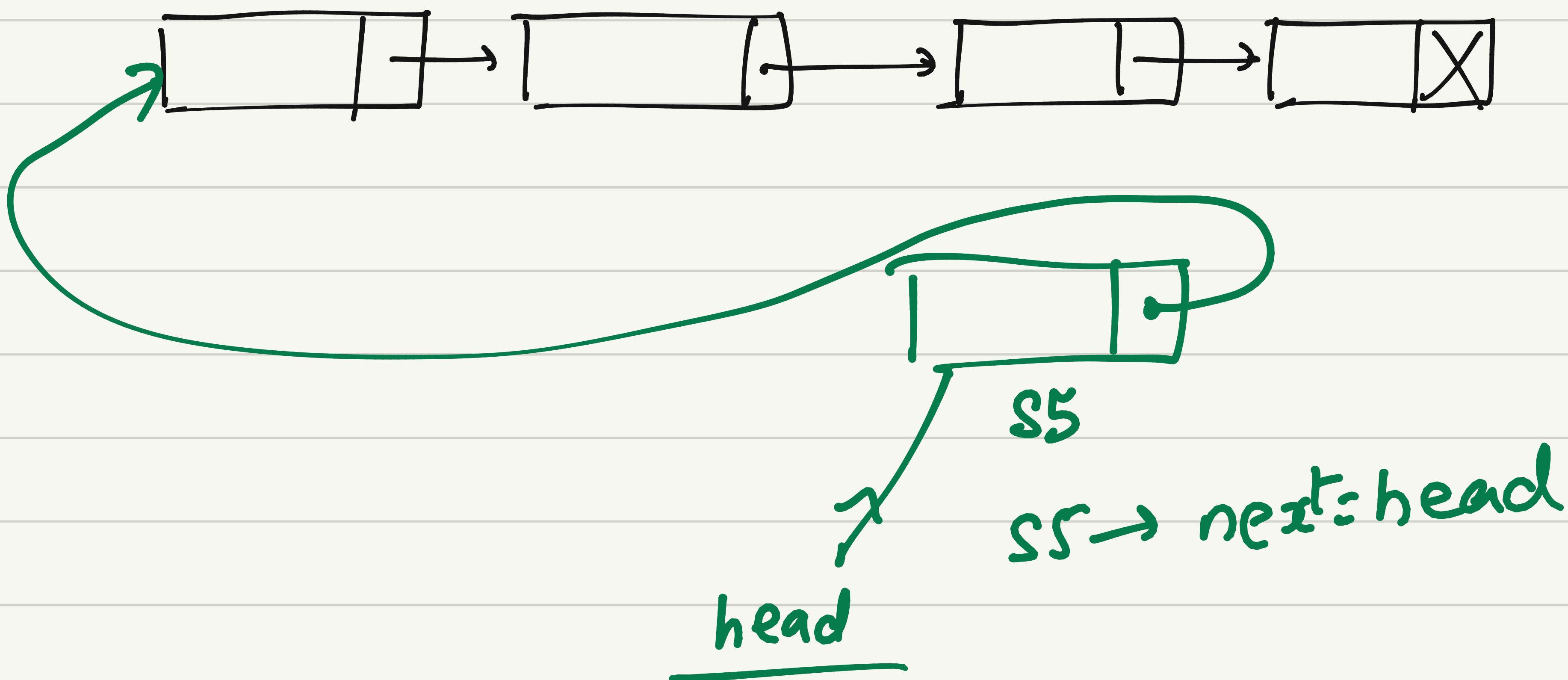
S4 → rollno = " ",

S4 → name = " ",

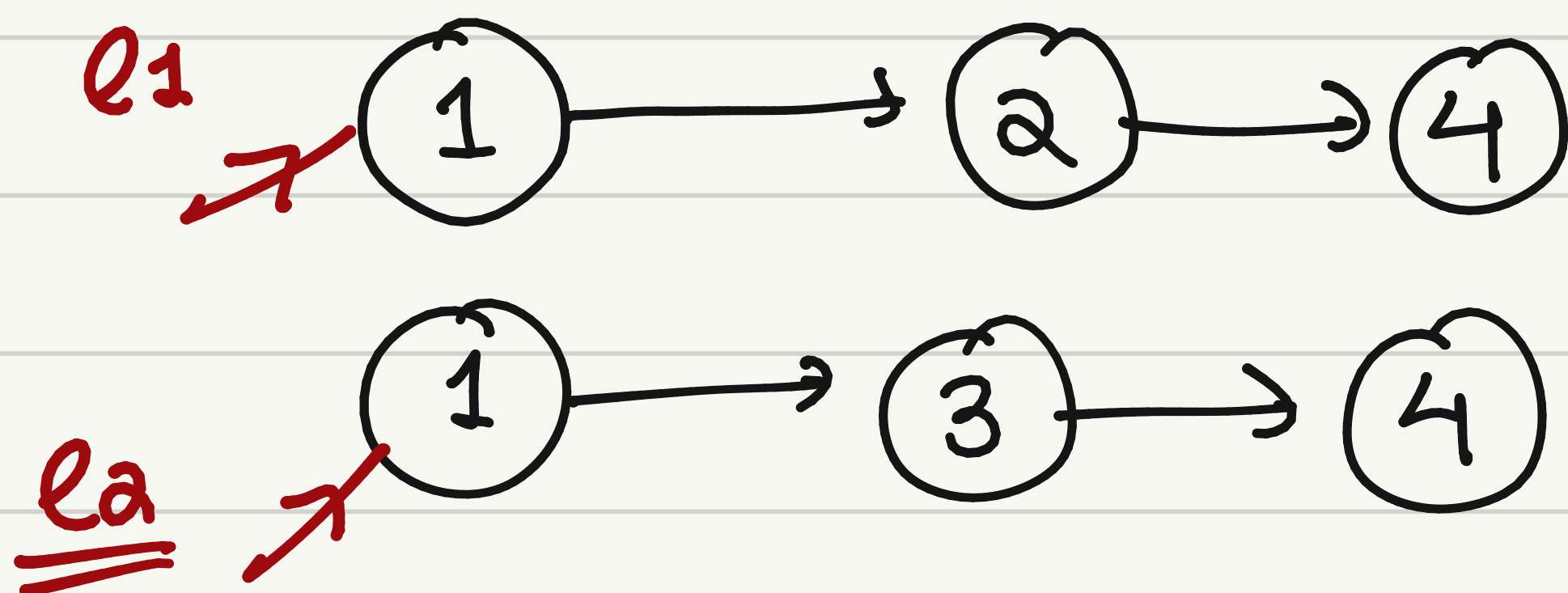
S4 → next = " null ",



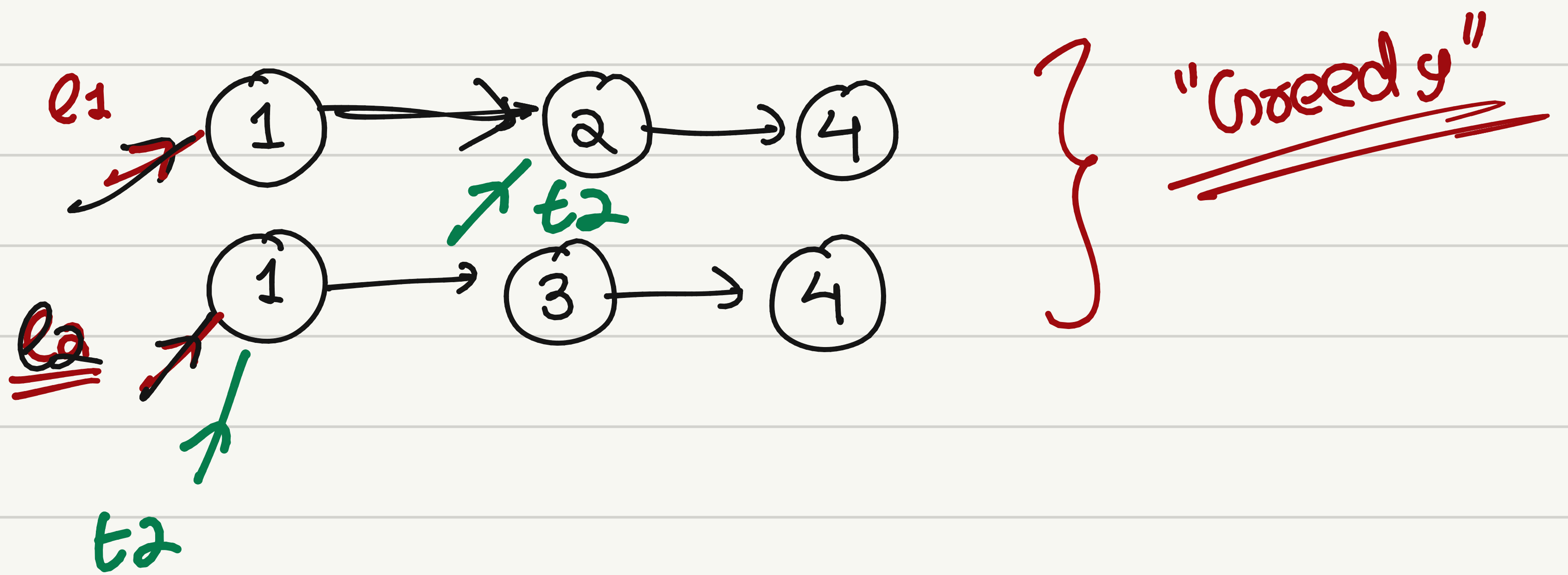
temp → next != NULL



21. Merge two sorted lists



"head → val"
"head → next"



$t1 = l1; \quad t2 = l2;$

$ListNode * temp;$

while ($t1 \neq null$ && $t2 \neq null$) {

if ($t1.val \leq t2.val$)

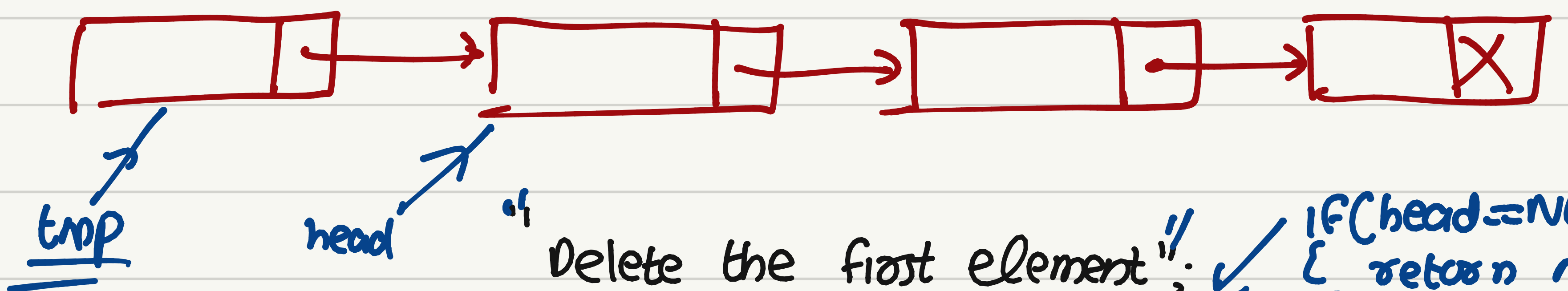
{
 $temp \rightarrow val = t1;$
 $t1 = t1 \rightarrow next;$

}

else
 {
 $temp \rightarrow val = t2;$
 $t2 = t2 \rightarrow next;$

}

}



"Delete the first element";

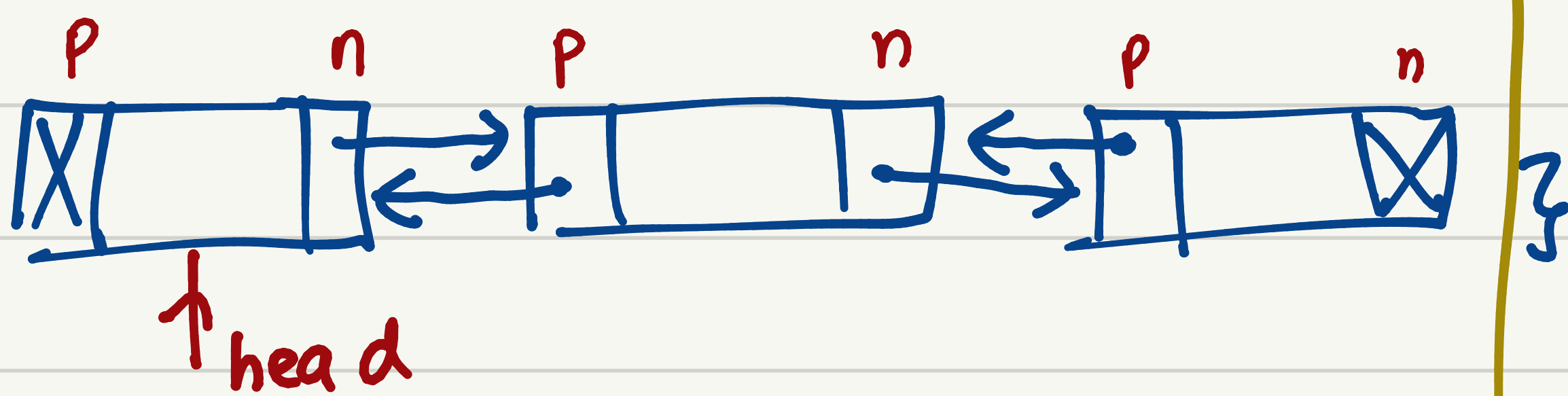
if ($head == null$)
 {
 $return null;$
 }

1) Student $*tmp = head;$

2) $head = head \rightarrow next;$

$free(tmp);$

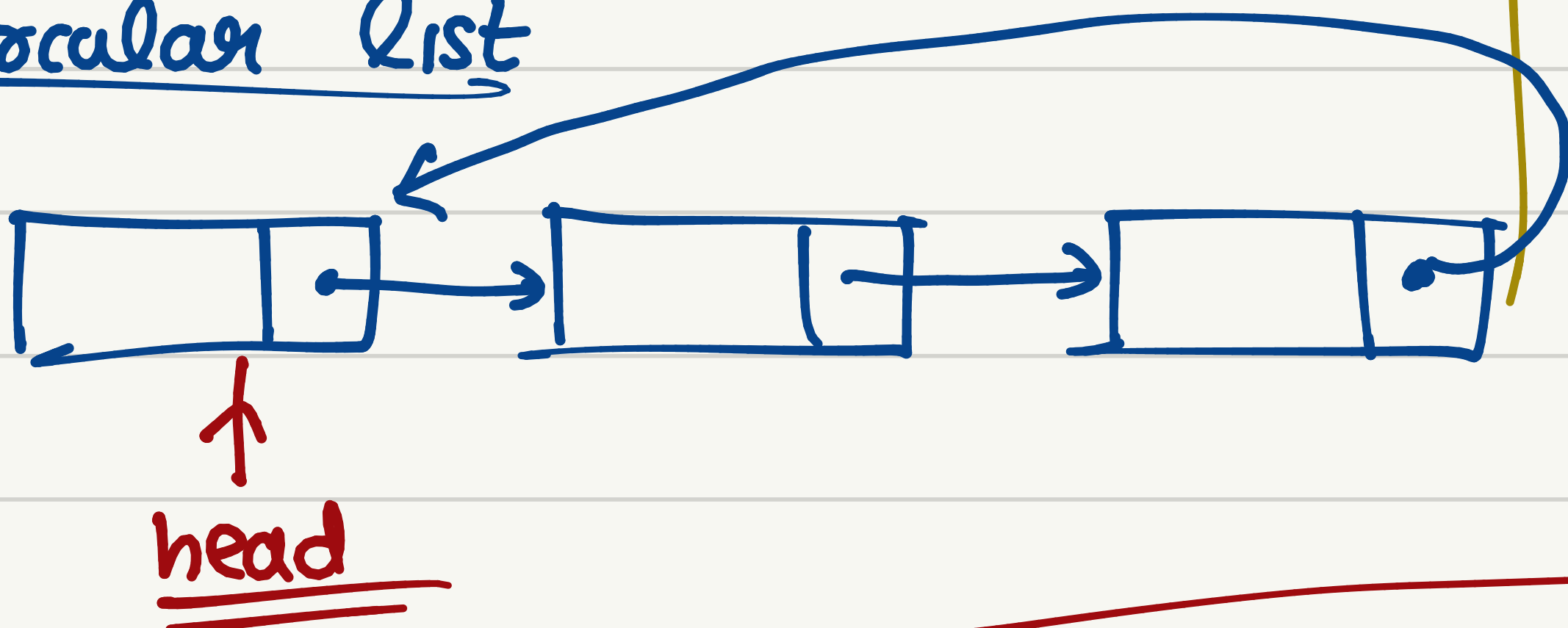
Double Linked List



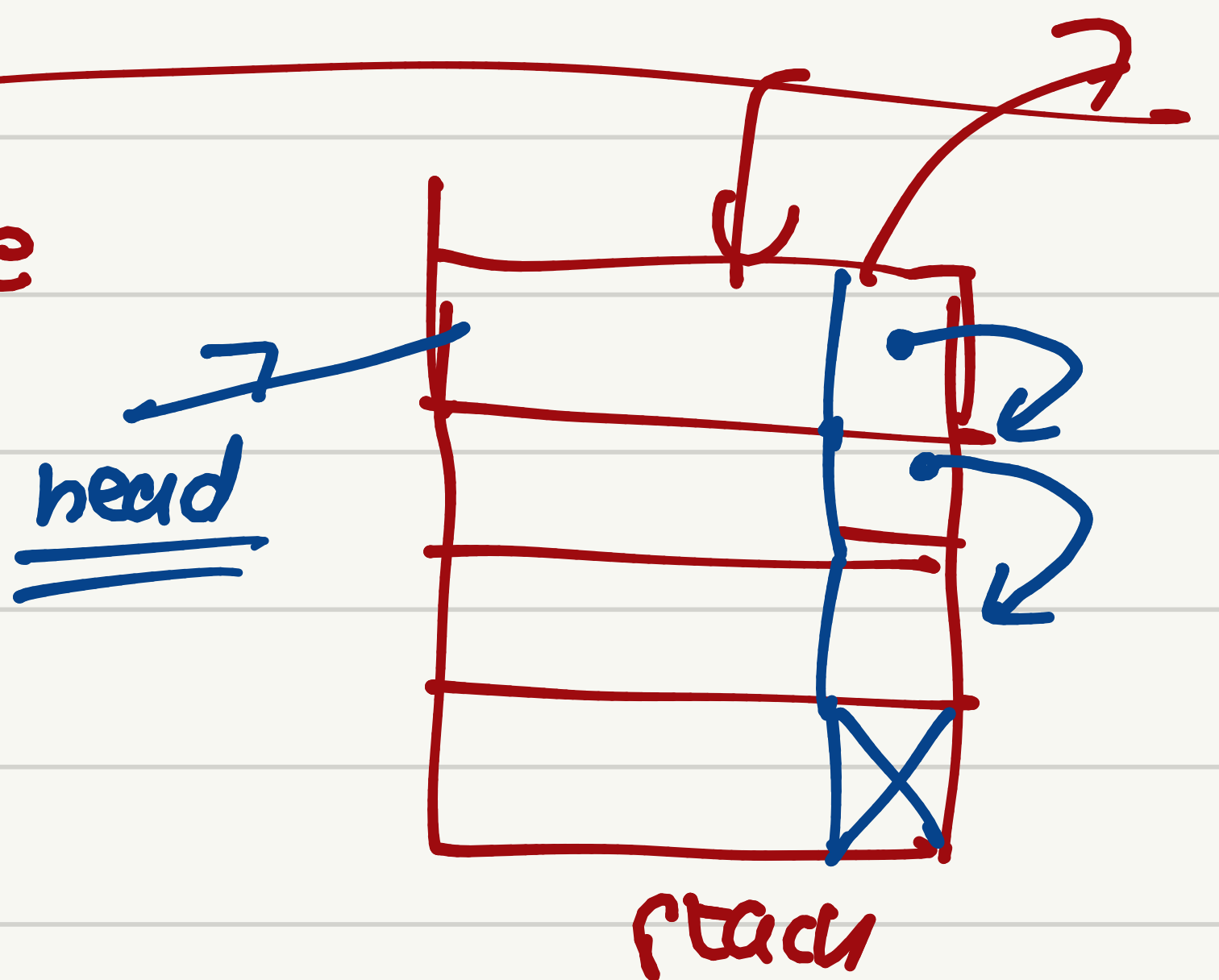
struct A {

struct A* next;
struct A* prev;

Circular List



Stack, Queue, Tree



factorial ()
{

$$\text{factorial}(n) = n * \text{factorial}(n-1);$$

Recursion

↳ When a solution to a problem depends on a solution to a subproblem of same nature
↳ "Recursion",

Sum of natural number numbers; n

$$n=1$$

$$1 = 1$$

$$n=2$$

$$1 + 2 = 3$$

$$n=3$$

$$1 + 2 + 3 = 6$$

$$n=4$$

$$1 + 2 + 3 + 4 = 10$$

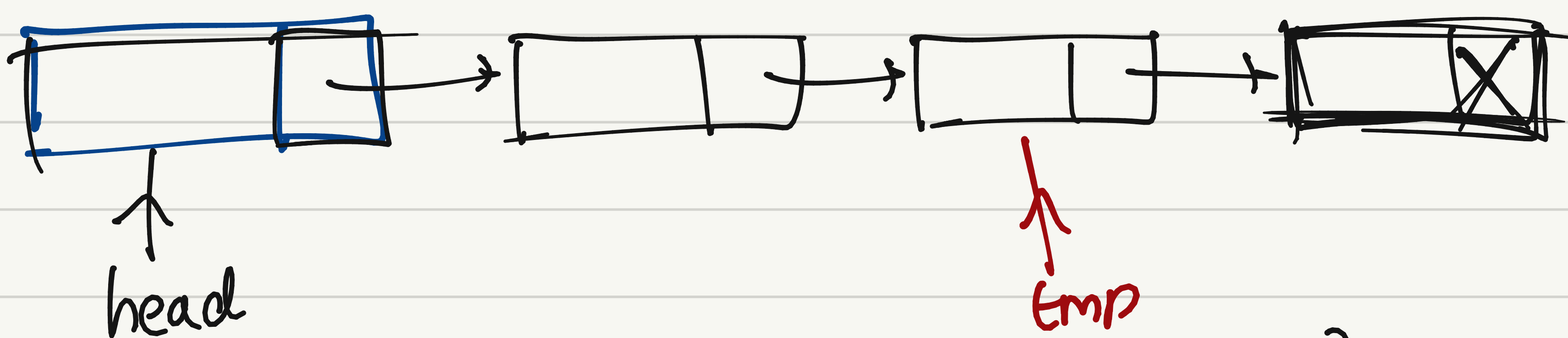
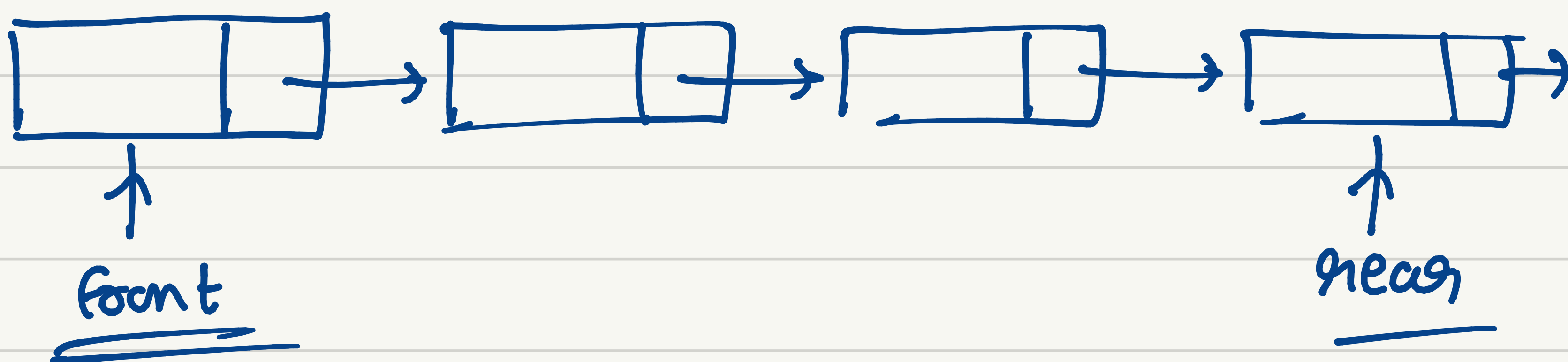
$f(n)$

Sum of n natural numbers = n

+ sum of n-1 natural numbers

$$f(n) = n + f(n-1) \quad ; \quad \text{Termination condition Recursive stop.}$$

Queue front, rear



$(tmp \rightarrow next) \rightarrow next = NULL;$
 $\{$
 $last = tmp \rightarrow next;$
 $tmp \rightarrow next = NULL;$
 $free(last);$
 $\}$

"231. LeetCode"

Recursive program

$$\begin{aligned}
 "n" &= 2^x \\
 24 &/ 2 = 12 \leftarrow 2
 \end{aligned}$$

$$\begin{aligned}
 24 &= 4 \times 3 \times 2 \\
 &= 2^3 \times 3^1 \times 2^1 \\
 &= 2^3 \times 3^1
 \end{aligned}$$

$$\begin{aligned}
 n &= 2^y \\
 &= 2^{n_1 + n_2} \quad n_1 + n_2 = y \\
 &= \underline{2^{n_1}} \times \underline{2^{n_2}}
 \end{aligned}$$